

Evulator

Genomic Computation: Theory, Models, and Systems

Canonical deep edition

Chapters 1–2

Hermon AI / coordinated research-textbook series

Development manuscript / 8 September 2026

About this development edition

This cumulative edition contains Chapters 1 and 2, not a completed book. Chapter 1's source and previously published PDF are preserved. The new chapter adds mechanisms, original vector figures, exact mathematical examples, executable references, exercises and worked solutions.

The intended standard is depth with straightforward explanation, without an artificial educational-level ceiling. Figures have editable SVG sources and semantic TXT companions. The repository also includes static state-changing keyframes, not exported moving media.

No new laboratory experiment, trained DOGMA or Hermon DNA result, GPU training run or inference-performance benchmark is reported. Computed teaching examples are explicitly distinguished from historical evidence and measured scientific results.

Review is author-agent source, mathematical, skeptical and rendered-page review, not independent specialist certification. Independent review and observation of real readers remain open. This PDF is not tagged and is not claimed to satisfy PDF/UA.

Contents

1	Why Genomic Computation?	1
1.1	Three ways to organize computation	1
1.2	Stored, expressed, and adapted are different roles	2
1.3	What biological regulation actually contributes	3
1.4	An exact typed example of expression	4
1.5	Development: constructing the working system	6
1.6	Execution, learning, and structural change	7
1.7	Before calling it genomic, check computer science	8
1.8	Evolutor, DOGMA, and Hermon DNA	9
1.9	Two memory strategies, one honest budget	11
1.10	A research program with concrete stopping points	12
1.11	Exercises	13
1.12	Worked solutions and discussion	14
2	Learning objectives, data, tasks, and evaluation	15
2.1	Five objects that a score cannot identify	15
2.2	A learning problem includes a population and a procedure	16
2.3	Autoregression is an information contract	16
2.4	Tokenization changes the prediction problem	17
2.5	Data families answer different questions	18
2.6	Split relationships before splitting rows	19
2.7	The random-data entropy control	20
2.8	Intervene on future tokens	20
2.9	Why gradients alone are not enough	22
2.10	A CPU reference experiment with a known failure	22
2.11	Capacity is a vector of confounds	23
2.12	Baselines, removal tests and transplants	24
2.13	Seeds reveal outcomes, not just a mean	25
2.14	Loss, perplexity and bits: keep the denominator	25
2.15	Accuracy, calibration and systems measurements	26
2.16	Exact synthetic tasks before learned solutions	27
2.17	Balanced memory tasks and shortcut audits	28
2.18	Budgets and the experiment lifecycle	29
2.19	Exercises	30

2.20 Worked solutions	31
Chapter 1 glossary	33
Chapter 2 glossary	34
Bibliography	35
Index	37

CHAPTER 1

Why Genomic Computation?

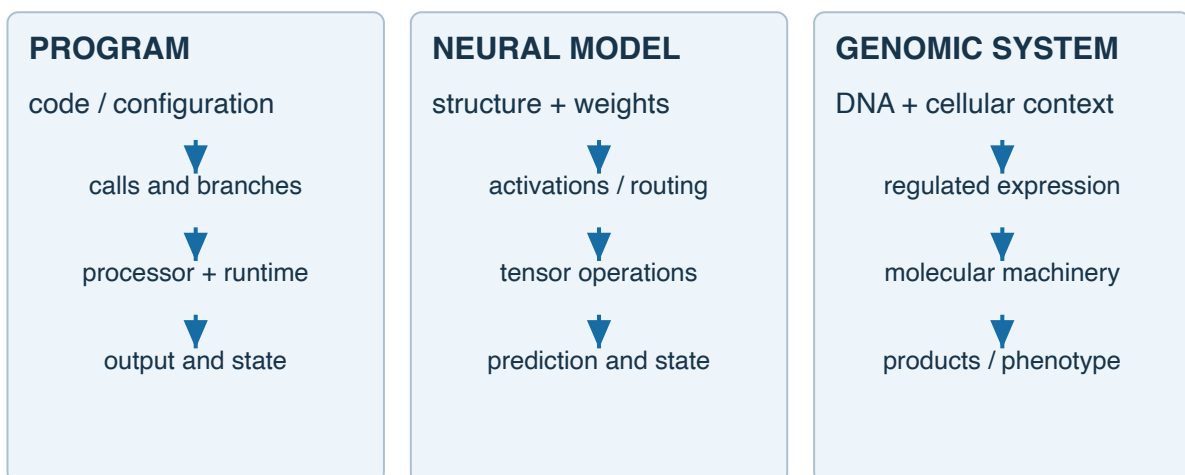
The research question

A genome can persist while different parts participate in different activities. Can an engineered separation between stored structure, expressed computation, and structural adaptation yield a useful computational organization beyond ordinary routing and program construction? This chapter makes the question precise without assuming the answer.

1.1 Three ways to organize computation

Imagine a system asked to behave differently in two environments while retaining a common internal library. A program can branch. A neural model can change its activations or route work. A cell can alter gene expression. Each description involves persistent structure and context-dependent activity, but their mechanisms are not interchangeable.

The comparison becomes useful when we ask the same operational questions: what is stored, what executes, what selects the activity, what remains afterward, and what can change the stored organization? Figure 1.1 provides an entry map. The table below sharpens it without treating one column as a replacement for the others.



Aligned questions do not make the three mechanisms identical.

Figure 1.1: Three organizations compared through persistence, control, activity, and change. A program, a neural model, and a biological genomic system can all separate stored information from current activity. Similar roles do not imply identical mechanisms or equivalent capabilities.

Question	Program	Neural model	Biological genomic system
What persists?	Source, binary, configuration, saved data	Architecture, weights, optionally saved state	Genome; inherited and cellular context also matter
What executes?	Instructions and invoked routines	Numerical operations producing activations	Molecular processes involving RNA, proteins, and other components
What selects?	Branches, dispatch, scheduler, inputs	Activations, attention, gates, routing	Regulatory interactions in a cellular environment
What adapts?	Whatever update procedures are implemented	Parameters, state, or structure under specified update rules	Physiological and regulatory changes; population evolution across generations
What changes structure?	Code/configuration changes or generated programs	Architecture search, pruning, growth, redesign	Variation, inheritance, selection, and developmental processes

The biology column is an orientation, not a universal inventory of every organism. Gene expression can yield functional RNA or protein; control occurs at multiple stages, not only through an on/off switch at a DNA site.[1, 2] The software columns are likewise broad: conventional programs can be stateful, reflective, adaptive, or generated by other programs.

This immediately rules out a weak argument for Evolutor. Merely observing that stored components outnumber active components does not establish a new kind of computation. The research must identify a mechanism, a useful property, and a comparison under which that property matters.

1.2 Stored, expressed, and adapted are different roles

Let G denote a persistent computational description, c a context, x an input, and s a runtime state. A proposed abstraction separates selection, expression, and execution:

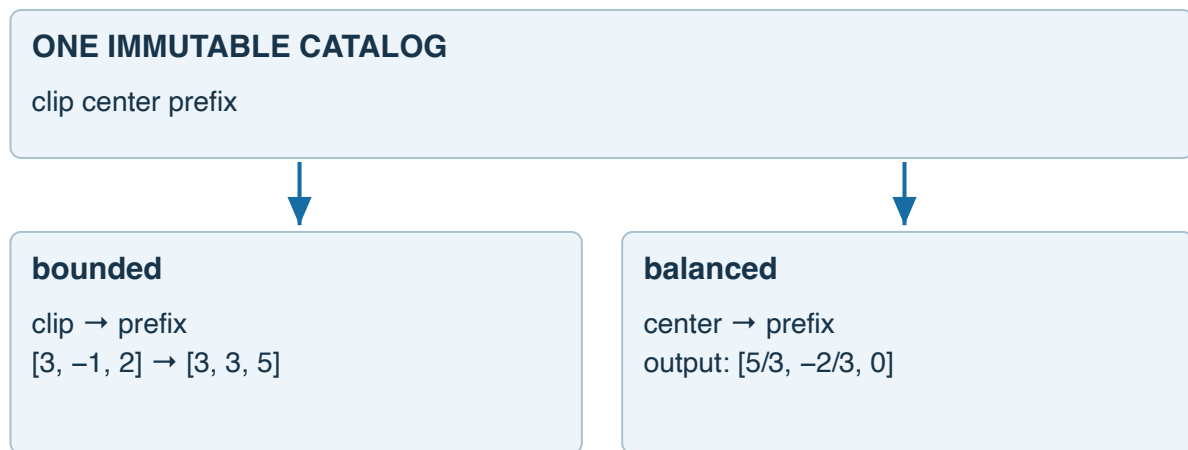
$$\begin{aligned}
 r &= \text{Reg}(G, c, s), \\
 P &= \text{Expr}(G, r), \\
 (y, s') &= \text{Exec}(P, x, s).
 \end{aligned}$$

Here r is a control decision, P is an executable plan, y an output, and s' the next state. These are candidate software interfaces, not equations of cell biology.

Stored structure describes what the system could use. **Expressed computation** is the selected or constructed activity for this occasion. **Adapted structure** is a later persistent description G' produced by an update process. The shorthand

$$\text{stored structure} \neq \text{expressed computation} \neq \text{adapted structure}$$

distinguishes roles and times, not values that must always be numerically unequal. An update may leave $G' = G$; an expression rule may activate the entire catalog.



Same catalog and input; context changes the selected plan.

Figure 1.2: One immutable catalog supports two plans for the same input. Only the context changes. This exact routed-program example illustrates stored versus active structure; it does not implement learning, biological expression, or a new architecture.

Separating these roles may help with inspection and controlled change. We can inspect a plan before execution, log which modules ran, and restrict what an update may alter. But those are also familiar benefits of interpreters, workflows, and modular software. The notation creates a place to ask questions; it does not answer the novelty question.

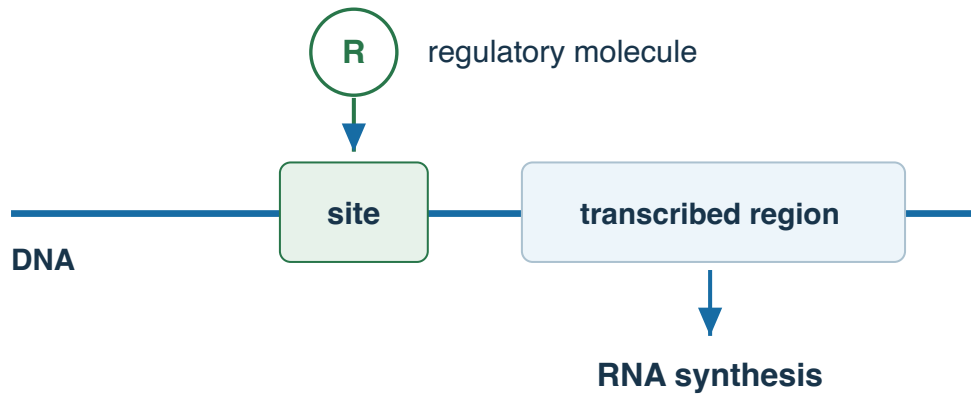
The crucial distinction is counterfactual. If we change the context while holding G fixed, we test regulation. If we change G and replay the same context and input, we test persistent structural change. If neither intervention is possible or observable, describing the system as adaptive may hide rather than clarify its behavior.

1.3 What biological regulation actually contributes

A genome is not a list of Python functions. Its sequence participates in a cell that already contains molecular machinery, spatial organization, and a history. Many differentiated cells can carry substantially the same genome while exhibiting different expression patterns; exceptions such as immune-cell DNA rearrangements prevent an absolute equivalence claim.[2, 3]

Gene expression produces functional products from genomic information. Transcription produces RNA; some RNAs function directly and others serve as templates for protein production.[1] Regulation can affect transcription, RNA processing or stability, translation, and protein activity.[2] Thus a useful computational analogy must not reduce expression to fetching one stored object unchanged.

A qualitative transcription-control example



Binding can change activity; the DNA sequence need not change.

Other control points exist; no quantitative rates are shown.

Figure 1.3: A qualitative transcription-control sketch. A regulatory molecule interacts with a DNA-associated site and can alter activity without rewriting the DNA sequence. The figure omits detailed complexes, kinetics, and other control points; the arrow means an interaction, not a software call.

The computational question extracted from this biology is narrower than a claim of imitation: can a persistent description support context-sensitive production of working structure with tractable rules for reuse, interference, and change? In software, we are free to design that interface differently from a cell. What we may not do is use the biological origin of an analogy as evidence of engineering performance.

A switch choosing one of two branches is a valid model of one control decision. It is not a quantitative gene-regulatory network. To justify that stronger description, we would need states, interactions, rates or transition rules, an account of feedback, and evidence that the abstraction preserves the biological property of interest. Chapter 1 introduces the distinction; later biological chapters develop the mechanisms.

1.4 An exact typed example of expression

We now build a deliberately ordinary program. Its simplicity is useful: every change is visible, and it has an exact conventional baseline.

Let

$$\mathcal{X} = \bigcup_{m \geq 1} \mathbb{Q}^m$$

be the type of nonempty finite rational sequences. For $x = (x_1, \dots, x_m)$ define three length-preserving modules:

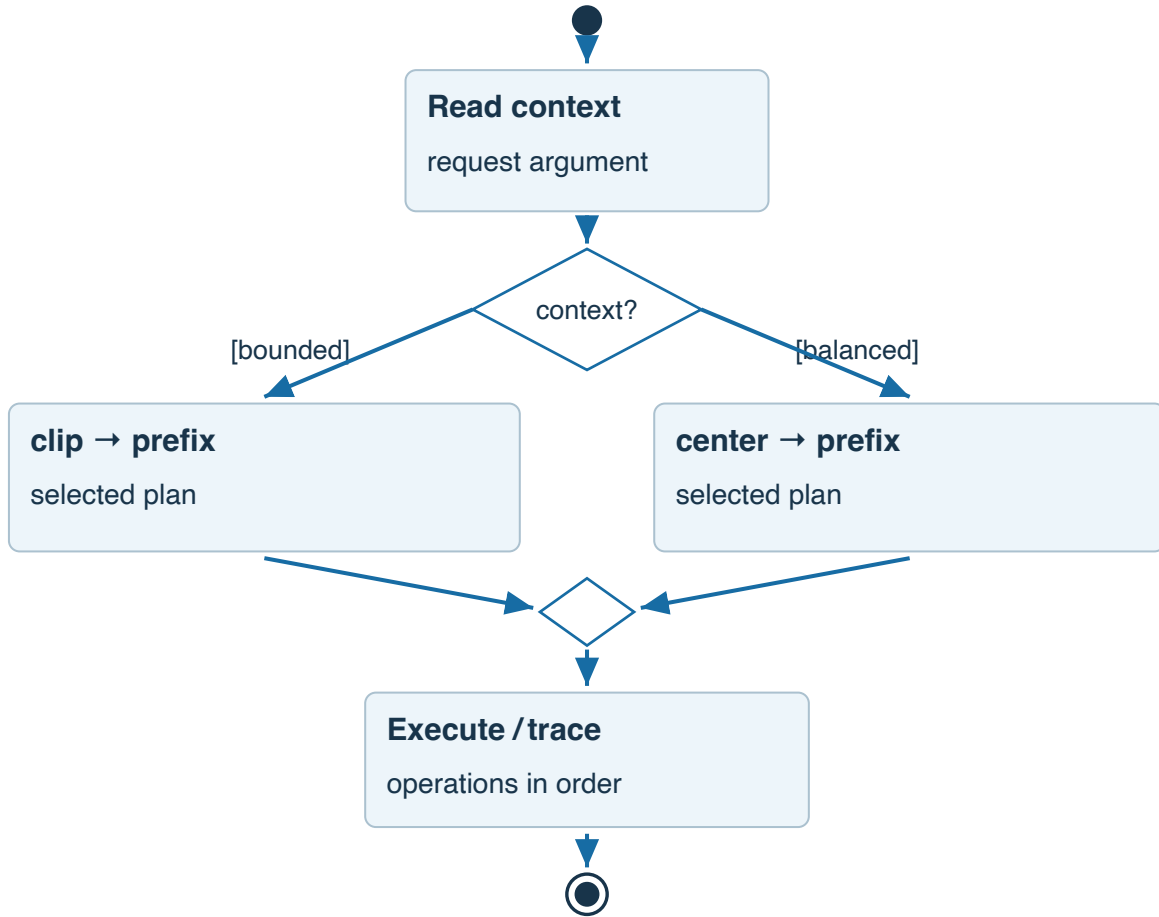
$$\begin{aligned} \text{clip}(x)_i &= \max(0, x_i), \\ \text{center}(x)_i &= x_i - \bar{x}, & \bar{x} &= \frac{1}{m} \sum_{j=1}^m x_j, \\ \text{prefix}(x)_i &= \sum_{j=1}^i x_j. \end{aligned}$$

Each module has type $\mathcal{X} \rightarrow \mathcal{X}$. A plan is a finite ordered list of module names. Its meaning is their left-to-right composition. The shared type makes every such composition well-typed; it does not make every composition useful.

The context set is $\mathcal{C} = \{\text{bounded}, \text{balanced}\}$. Our selection rule is

$$\text{Reg}(c) = \begin{cases} (\text{clip}, \text{prefix}), & c = \text{bounded}, \\ (\text{center}, \text{prefix}), & c = \text{balanced}. \end{cases}$$

The catalog is immutable. There are no learned parameters. Context is an explicit argument, not inferred from hidden state.



UML activity; unknown context raises an error (not shown).

Figure 1.4: UML activity-style control flow for the worked example. Guarded alternatives select one ordered plan; execution records intermediate values. Unknown contexts raise an error outside the two successful branches shown. This is software control flow, not a biochemical pathway.

For $x = (3, -1, 2)$, the bounded route gives

$$(3, -1, 2) \xrightarrow{\text{clip}} (3, 0, 2) \xrightarrow{\text{prefix}} (3, 3, 5).$$

For the balanced route, $\bar{x} = 4/3$, so

$$(3, -1, 2) \xrightarrow{\text{center}} (5/3, -7/3, 2/3) \xrightarrow{\text{prefix}} (5/3, -2/3, 0).$$

Exact rational arithmetic makes these results transparent. The generated table comes directly from the executable source:

Context	Plan	Output
bounded	clip / prefix	3, 3, 5
balanced	center / prefix	5/3, -2/3, 0

The two contexts express different mathematical invariants. The bounded output is nonnegative and nondecreasing because it sums nonnegative terms. The balanced output ends at zero because the centered values sum to zero. Neither invariant says the intermediate balanced prefix must be nonnegative. The negative second entry is correct.

Order matters. Applying prefix before clip to $(3, -1, 2)$ yields $(3, 2, 4)$, not $(3, 3, 5)$. A set of active names cannot represent this distinction; an ordered plan can. This is the first reason to specify expression semantics more precisely than an active/inactive mask.

The source below records both input and output of each operation. The excerpt is generated from the tested file, `examples/deep/ch01.py`, not independently retyped.

```

20 def select(context):
21     if context=="bounded":
22         return ("clip","prefix")
23     if context=="balanced":
24         return ("center","prefix")
25     raise ValueError("unknown context")

27 def execute(plan, values):
28     xs=tuple(Fraction(x) for x in values)
29     if not xs:
30         raise ValueError("nonempty rational sequence required")
31     trace=[]
32     for name in plan:
33         if name not in CATALOG:
34             raise ValueError("unknown module")
35         before=xs
36         xs=CATALOG[name](xs)
37         trace.append({"module":name,"before":before,"after":xs})
38     return xs,trace

```

A direct branching implementation computes the same specification without a plan interpreter. Tests compare the two implementations over a grid of rational inputs, not just the displayed example. We can therefore say the abstraction exposes control structure; we cannot say that it enables a function ordinary software could not compute.

1.5 Development: constructing the working system

Expression selects or produces activity for a context. **Development** asks a related but distinct question: how is an operational system constructed from a more compact persistent description?

Biological development mediates the relation between genotype and phenotype; it is not a static one-to-one lookup from a DNA string to a finished organism.[3] Environmental influences can affect gene expression during development.[4] An engineering analogy should therefore make the construction process explicit:

$$P = \text{Dev}(G, E).$$

Here G is a software description, E a construction environment, and P a program. Unlike the organism, this particular P is a digital artifact with defined execution semantics. In a richer

model, development could depend on time, feedback, and stochastic events; the equation alone supplies none of them.

Our minimal generator takes a positive integer k and constructs a plan containing k copies of prefix. For $k = 2$, the input $(3, -1, 2)$ becomes $(3, 2, 4)$ after the first pass and $(3, 5, 9)$ after the second.



Generating a program is already possible in ordinary software.

Figure 1.5: A compact description constructs an ordered program before that program processes data. Repeating prefix twice yields the displayed exact trace. This original program generator is an analogy for description-to-structure construction, not a model of embryogenesis.

The construction already exists in ordinary programming. Its research value would have to come from how descriptions compose, how variation changes their products, or what constraints the development process enforces. A small description might generate a large repeated structure, but executing that structure still costs time and memory. Compression of description is not free execution.

This example also separates validity from fitness. Every positive k produces a well-typed plan, but increasing k need not improve a task objective. A type checker can reject malformed structure; it cannot replace an experiment measuring whether the structure solves the intended problem.

1.6 Execution, learning, and structural change

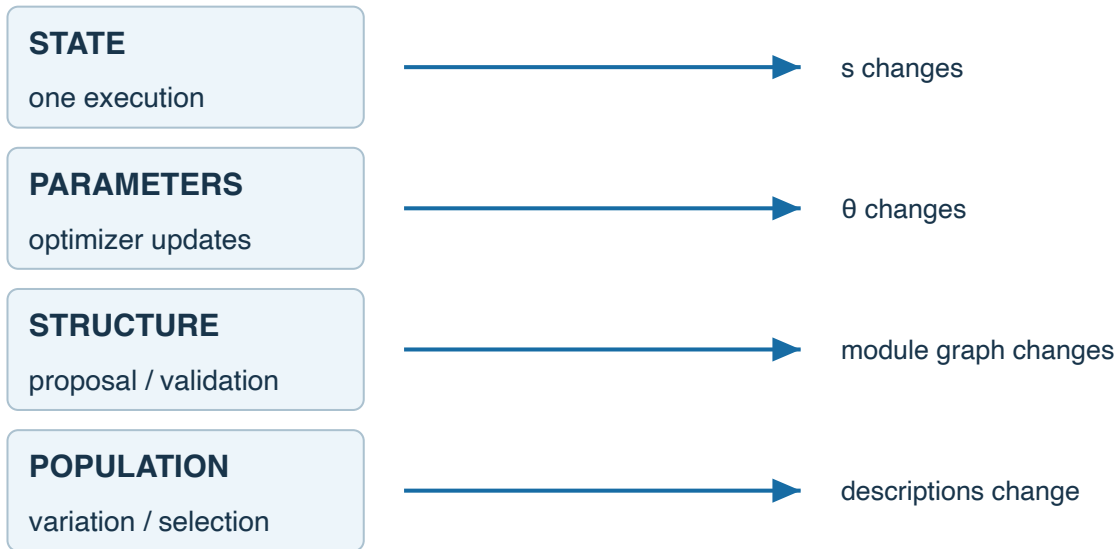
The same system can contain several update processes. A runtime state can change on every input while parameters remain fixed. An optimizer can change parameters while topology remains fixed. A structural procedure can add a module, alter a connection, or replace a description.

A candidate formal separation is

$$\begin{aligned}
 (s_{t+1}, y_t) &= F_{G, \theta}(s_t, x_t), \\
 \theta_{j+1} &= U(\theta_j, G, \mathcal{D}_j), \\
 G_{k+1} &= A(G_k, \mathcal{E}_k).
 \end{aligned}$$

The indices deliberately differ. t counts execution steps, j parameter updates, and k structural proposals. $\mathcal{D}_j$ is learning data and $\mathcal{E}_k$ the evidence used to accept, reject, or revise a proposal.

These equations describe possible interfaces; the chapter’s exact program implements neither U nor A .



Different update rules; no universal ordering of physical durations.

Figure 1.6: Four update roles: state, parameters, structure, and a population of inherited descriptions. Arrow meanings are update relations. No numerical biological or computational duration is asserted, and the rows do not impose a universal speed ordering.

One might run many inference steps between optimizer updates and many training runs between structural changes. That schedule is a design choice, not a theorem. Online learning can update parameters on every sample; program generation can happen in a millisecond; structural search can run offline for days.

An important engineering issue appears as soon as G changes: what happens to s and θ ? If the new structure has different state dimensions, blindly restoring an old checkpoint is invalid. A structural transition needs a migration map, a reset policy, or rejection. For example, replacing a state vector in $\mathbb{R}^d$ with one in $\mathbb{R}^{d'}$ requires specifying a map $\mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ or explicitly discarding the old state.

This is where the genome analogy may become productive. An architecture that makes variation, construction, and migration explicit could be easier to reason about than ad hoc topology changes. Whether that organization improves learning or robustness is a separate empirical question.

1.7 Before calling it genomic, check computer science

The novelty burden is substantial because the neighboring fields are rich. Dynamic dispatch selects behavior from persistent code. Plugins extend a catalog. Query planning constructs an execution plan. Our exact example is already expressible with these ordinary ideas.

Mixture-of-experts systems route examples through selected subnetworks; Shazeer and colleagues’ sparse gate is a concrete established example.[5] Attention computes context-dependent weighted combinations of value representations from query–key scores.[6] Neither activity requires persistent architecture modification.

Neural architecture search can generate and evaluate model descriptions, as in Zoph and Le’s reinforcement-learning construction.[7] Program synthesis seeks a program meeting a specification;[8] genetic programming uses evolutionary search over programs.[9] These are direct neighbors of computational development and structural adaptation, not optional footnotes.

State-space models provide another established way to organize sequence computation. Mamba makes selected state-space parameters depend on the input so the recurrence can retain or discard information conditionally.[10] Agent systems also already compose reasoning with actions that obtain new information; ReAct provides one concrete example.[11]

Dispatch / plugins	select stored routines
MoE / conditional networks	route work among experts
Attention / retrieval	query accessible representations
Recurrence / SSMS	update carried state
Synthesis / architecture search	construct or search programs
Agent / workflow systems	compose tools and actions

A new label must earn a new semantic or empirical contribution.

Figure 1.7: Established alternatives organized by the role they already serve. The same proposed genomic mechanism may need comparison with several rows. This is a conceptual comparison map, not a benchmark or an exhaustive taxonomy of computer science.

A serious proposal should now answer a specific question. Does it offer a new semantics for compositional change? A useful inductive bias? A resource advantage on a defined task family? Better isolation of modifications? Or only a helpful vocabulary? Any of these may be worth studying, but they are different contributions.

For our two-context example, the direct baseline is functionally equivalent. If the plan interpreter is slower, a claim of inference efficiency fails for that implementation. It may still improve inspectability because the plan and trace are explicit artifacts. Such a result would support a tooling argument, not a new learning architecture.

A future comparison should match training data, task distribution, quality criterion, and relevant resource budgets. An ablation removing the proposed regulation mechanism should preserve other capacity as far as possible. Otherwise a larger catalog or extra training may explain the gain. This is the minimum experiment needed to turn a naming proposal into an engineering question.

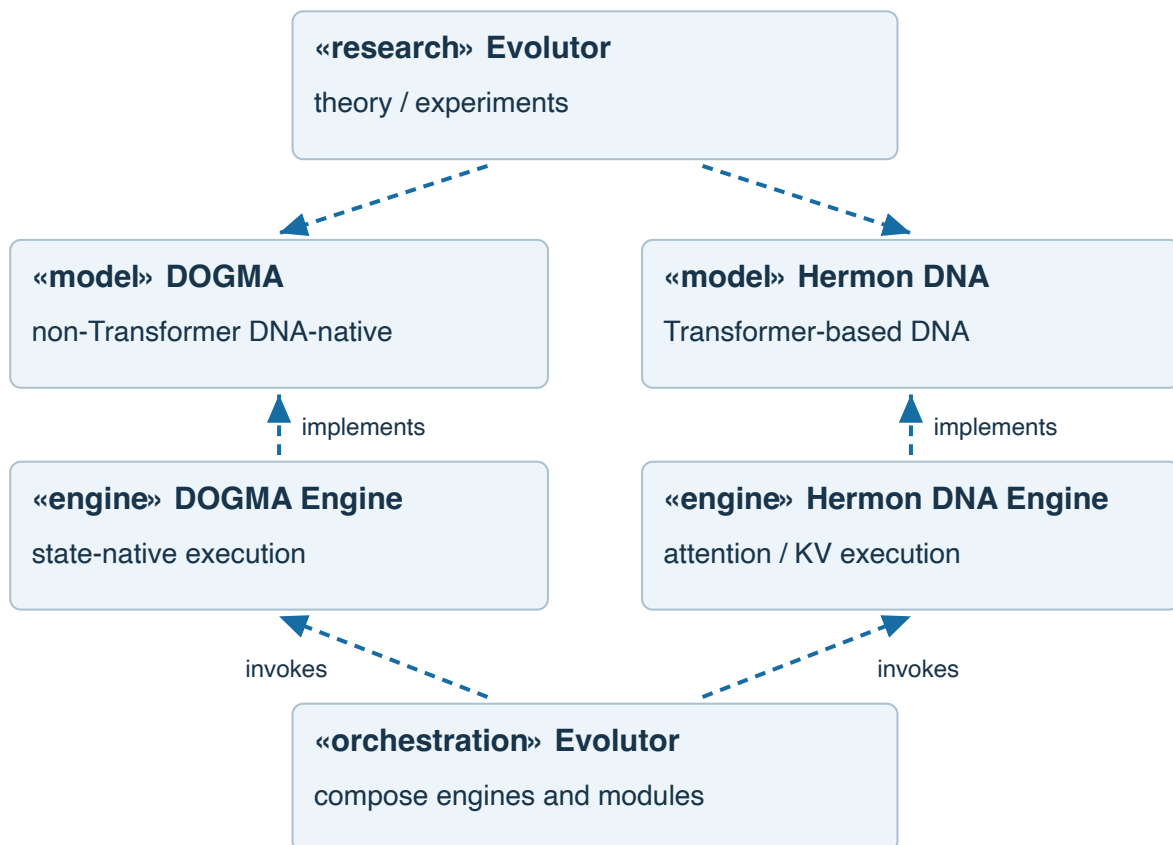
1.8 Evolutor, DOGMA, and Hermon DNA

This book uses three project levels with fixed meanings.

Evolutor is the broader genomic-computation theory and research framework, including the planned compiler/runtime and orchestration layer above model families. It may eventually coordinate models, retrieval, symbolic modules, and tools. That is a design hypothesis, not a deployed capability established here.

DOGMA names the non-Transformer DNA-native architecture research line and its corresponding non-Transformer inference engine, DOGMA Engine. The opening hypothesis emphasizes carried state, regulation, and expression.

Hermon DNA names the Transformer-based DNA architecture research line and its corresponding Transformer inference engine, Hermon DNA Engine. Its opening hypothesis retains attention and key/value context representations.



Dashed arrows: client depends on target; proposed roles only.

Figure 1.8: UML component-style research taxonomy. Evolutor contains the research and orchestration roles above the two families. DOGMA and Hermon DNA have separate model and engine identities. Dashed arrows express design relationships, not deployment or verified superiority.

In this project vocabulary, DNA-native and DNA-aware are research intentions whose operational content must be supplied by later representations, objectives, and mechanisms. They do not imply that either model runs in a test tube. Physical DNA computing, genomic sequence modeling, and biologically inspired digital organization must remain distinguishable.

An architecture specifies computations and state. An engine implements those computations under a resource and numerical contract. The engine may optimize layouts or kernels without

changing the intended model. Conversely, a changed state transition is an architectural change, even if introduced inside an implementation.

This distinction prevents a roadmap from collapsing into a list of products. We first need formal semantics and a reference model, then tests of equivalence, then evidence from experiments, then justified optimization. Naming an engine before those steps does not make it available.

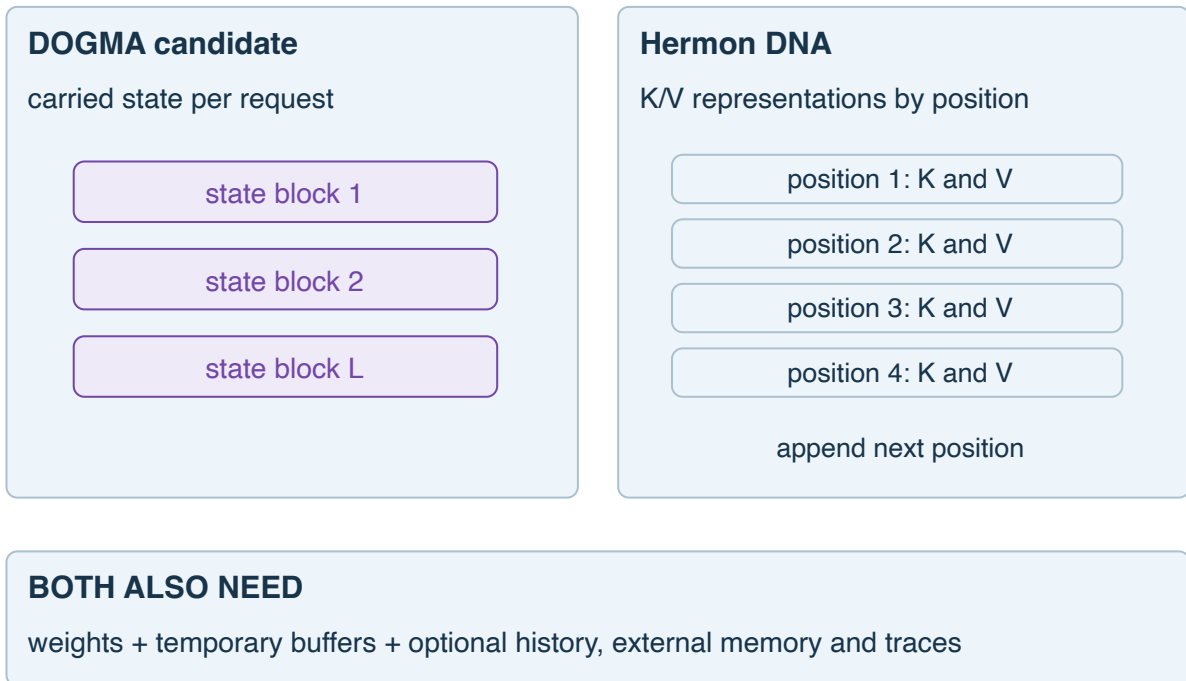
1.9 Two memory strategies, one honest budget

A recurrent candidate can summarize prior input in carried state:

$$(s_{t+1}, y_t) = F_\theta(s_t, x_t).$$

The state is sufficient only for the behavior the model can actually support. With fixed finite precision, a bounded state cannot preserve arbitrarily many distinguishable histories without collisions.

A causal attention model instead can retain key/value representations indexed by past position. Let C_{t-1} contain the cached pairs for positions $1, \dots, t-1$. Processing x_t appends (k_t, v_t) , forms C_t , and computes an output using the permitted prefix. The indexing matters: using a representation from a future token would violate the causal contract.



Compression and addressability are strategies, not a universal ranking.

Figure 1.9: Two candidate per-request memory organizations. Carried state transforms a bounded representation; a conventional full-context KV cache retains position-indexed representations. Both also require weights and workspace and may add external memory, traces, or history. The figure does not claim equal capability.

Attention's basic operation is

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathcal{M}\right) V,$$

where $Q \in \mathbb{R}^{q \times d_k}$, $K \in \mathbb{R}^{T \times d_k}$, and $V \in \mathbb{R}^{T \times d_v}$. The mask $\mathcal{M}$ excludes forbidden positions, and softmax acts across the T key positions for each query. This is an established mechanism, not a proposed DOGMA operation.[6]

Now derive a deliberately hypothetical storage comparison. Suppose batch size is B , layer count L , retained positions T , key/value head count H_{kv} , head width d_h , and storage per scalar b bytes. A conventional uncompressed KV layout needs

$$M_{KV} = 2BLTH_{kv}d_hb.$$

The factor two counts keys and values. A design storing d_s carried-state scalars per layer instead needs

$$M_s = BLd_sb.$$

These formulas count only the declared persistent per-request arrays. They omit weights, temporary activations, allocator overhead, retrieval stores, and any additional recurrent history.

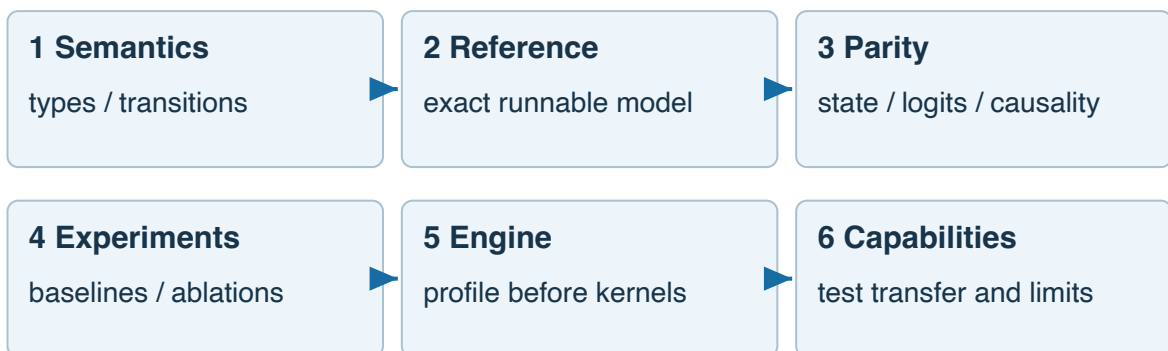
Take $B = 1$, $L = 24$, $T = 4096$, $H_{kv} = 8$, $d_h = 64$, $d_s = 2048$, and $b = 2$. The exact helper returns 201,326,592 bytes for KV, or 192 MiB, and 98,304 bytes for state, or 96 KiB. Here MiB and KiB mean powers of 1024. The array-size ratio is 2048.

That ratio is *not* a measured speedup and does not compare equal-quality models. A smaller state may discard distinctions a task needs. A cache may be windowed, quantized, shared, compressed, or supplemented by other memory. DOGMA is not proved superior by substituting a chosen state width into an accounting formula.

The productive research question is conditional: for a specified task and error tolerance, how much information must remain accessible, how is it updated or addressed, and what does that cost? Both families exist to explore different answers rather than to predeclare a universal winner.

1.10 A research program with concrete stopping points

The worked program has already taught several useful distinctions. A catalog differs from a plan. A plan differs from its execution trace. Generating a larger plan differs from learning it. Changing runtime values differs from changing persistent structure. These are ordinary, precise concepts on which a more ambitious proposal can be built.



A failure sends the proposal back for revision; no gate is proof of AGI.

Figure 1.10: A research sequence from semantics to evaluated capabilities. Reference behavior precedes engine optimization; parity checks precede performance claims. Failure at a gate sends a proposal back for revision. These are planned investigations, not completed model or AGI results.

The next steps should preserve that precision. Define a state transition before naming a stateful model. Define what regulation may inspect before claiming causal inference. Define how development preserves types before allowing generated structure to execute. Define migration and evaluation before treating a structural update as improvement.

Evolutor ultimately studies mechanisms relevant to memory, modularity, adaptation, continual learning, structural learning, tool composition, and generalization. Those motivations connect to broad questions about intelligence, but Chapter 1 establishes no AGI capability. A useful negative result—for example, exact equivalence to a simpler routing baseline—can clarify which mechanisms deserve further work.

Book I supplies the molecular-computation foundation. Its new Chapter 1 teaches a particular graph-search construction and its limits; it does not supply all later biology, thermodynamics, and circuit prerequisites. The shared dependency plan remains explicit about future teaching. Readers may bring equivalent background, but a roadmap reference is not proof that a prerequisite chapter already exists.

Chapter 2 develops learning objectives, data, and evaluation: sequence likelihood, leakage-resistant splits, and held-out generalization. These foundations make later genomic proposals testable. It does not jump directly to an optimized engine. This chapter’s standard is the model for the rest of the book: use abstraction freely, make its semantics concrete, and explain the difficult part directly.

1.11 Exercises

1. Classify changing a context, changing a weight, adding a module, and recording a trace by the role each changes.
2. Compute both plans on $(-2, 4, 1)$ and verify their stated invariants.
3. Prove that the final entry of prefix after center is zero for every nonempty rational input. Does the proof require all other prefix entries to be nonnegative?
4. Find an input on which clip and prefix do not commute. Explain why an unordered active set is insufficient.
5. Describe a test that distinguishes a changed plan from a changed persistent catalog.
6. Run the development rule for $k = 3$ on $(3, -1, 2)$. Which properties are guaranteed by type compatibility, and which are not?
7. Propose a structural migration from $\mathbb{R}^d$ to $\mathbb{R}^{d+1}$. State what it preserves and what performance claim it does not justify.
8. Double retained context length T in the hypothetical memory example. Then double batch size. Which arrays change, and why is this not a throughput prediction?
9. Compare the exact routed example with its direct baseline. Formulate one legitimate tooling claim and one unsupported architecture claim.
10. Design a future ablation of a regulation mechanism, specifying a task, a matched baseline, an intervention, and a possible negative result.

1.12 Worked solutions and discussion

1. Context changes the selection input; a weight update changes parameters; adding a module changes stored structure; recording a trace changes an observation artifact. None alone establishes evolutionary adaptation.
2. Bounded: $(-2, 4, 1) \rightarrow (0, 4, 1) \rightarrow (0, 4, 5)$. Balanced: mean 1, then $(-3, 3, 0) \rightarrow (-3, 0, 0)$. The first is nondecreasing and nonnegative; the second ends at zero.
3. The final prefix is $\sum_i (x_i - \bar{x}) = \sum_i x_i - m\bar{x} = 0$. No sign constraint on partial sums follows; $(-3, 0, 0)$ supplies a counterexample to nonnegativity.
4. For $(3, -1, 2)$, prefix after clip is $(3, 3, 5)$, while clip after prefix is $(3, 2, 4)$. Both plans use the same two names, so order must be represented.
5. Snapshot the immutable catalog identity and contents, vary only the context, and compare selected plans and traces. A structural-change test instead versions the catalog and replays the same input and context.
6. The third prefix pass turns $(3, 5, 9)$ into $(3, 8, 17)$. Composition preserves nonempty rational sequence type and length. It does not guarantee bounded values, useful predictions, or improved performance.
7. $m(s) = (s, 0)$ preserves the first d coordinates. It does not preserve the full model's behavior unless the new transition and readout are also constrained appropriately.
8. With doubled T , KV becomes 384 MiB while the declared state remains 96 KiB. Doubling B then gives 768 MiB and 192 KiB. Runtime depends on computation, bandwidth, batching, and implementation, none measured by these formulas.
9. The plan and trace expose selected operations for inspection. That supports an inspectability claim for this implementation. It does not show a function inaccessible to ordinary software or a learning advantage.
10. One possible task is context-dependent sequence processing under a fixed total parameter and data budget. Replace learned regulation with a matched fixed routing rule while preserving modules; measure held-out error and resource costs. Equal performance would fail to support a benefit from the learned regulator on that task. It would not establish universal uselessness.

CHAPTER 2

Learning objectives, data, tasks, and evaluation

The central question

What information may a model use, what must it predict, and what observation would count against the proposed explanation of its success? We answer with exact indexing, deliberately broken predictors, relationship-aware splits, explicit budgets and executable reference tests before introducing new architectures.

2.1 Five objects that a score cannot identify

A model is not a task. A DNA corpus is not an objective. A low loss is not, by itself, evidence for the mechanism named in an architecture diagram. To make a meaningful claim, separate five objects: data, task, model, training objective, and evaluation protocol with its metrics.

Object	Concrete next-base example	Not interchangeable with
Data	sequence records + relationships	task
Task	predict next base from prefix	architecture
Model	prefix counts or trained θ	objective
Objective	mean negative log probability	success claim
Metric + protocol	held-out nats/base + split + mask	training loss alone

Figure 2.1: Five distinct objects in next-base prediction. A fixed predictor and a trained network can face the same task; one network can face several tasks. The score becomes evidence only after its data, information boundary, target construction and evaluation protocol are specified.

For example, a file of genomic sequences is data. Predicting the next nucleotide from the preceding prefix is a task. A smoothed prefix-frequency rule is a model, even though it has no learned weights. Mean negative log probability is an objective one could optimize. Mean held-out loss on a relationship-disjoint test set is an evaluation measurement. Changing any of these objects changes the question being answered.

DOMÉ organizes reporting around data, optimization, model and evaluation.[12] Our purpose is not to add forms for their own sake. It is to make experimental errors concrete

enough to reproduce. A predictor that reads the answer from a future input position can achieve excellent loss without learning the intended causal task; we will construct exactly that failure.

The taxonomy remains fixed throughout: DOGMA is a non-Transformer DNA-native architecture plus its non-Transformer engine; Hermon DNA is a Transformer-based DNA architecture plus its Transformer engine. Evolver is the broader theory, research, compiler, runtime and orchestration framework above both. This chapter reports no new trained implementation of either family.

2.2 A learning problem includes a population and a procedure

For supervised data $D = \{(x_i, y_i)\}_{i=1}^N$, let f_θ predict from x_i . An empirical objective is

$$\hat{R}_D(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i), \quad \theta^* \in \arg \min_{\theta} \hat{R}_D(\theta). \quad (2.1)$$

The parameter vector θ contains trainable quantities; $\mathcal{L}$ penalizes a prediction relative to its target. Optimization is a procedure intended to reduce the empirical objective. Equation 2.1 does not promise that an optimizer finds a global minimum or that such a minimum generalizes.

The population risk is instead $R(\theta) = \mathbb{E}_{(X,Y) \sim P}[\mathcal{L}(f_\theta(X), Y)]$. The distribution P includes how examples are sampled and what deployment population is intended. A held-out measurement estimates something about that population only if its sampling and information boundaries are appropriate. Overlapping windows from the same locus do not become independent biological examples merely because they occupy separate rows.

Regularization, augmentation, early stopping and hyperparameter search are part of the learning procedure. A test score evaluates the selected procedure, not an architecture name in isolation. Repeatedly choosing designs against a finite validation set can overfit that selection criterion;[13] looking at test scores while redesigning a model similarly turns the test into development information.

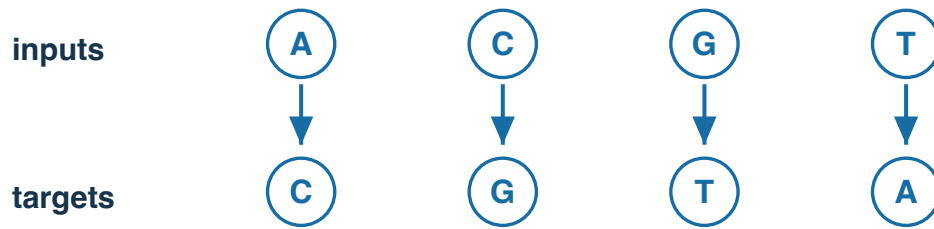
2.3 Autoregression is an information contract

For a sequence $x_1, \dots, x_T$,

$$p_\theta(x_{1:T}) = \prod_{t=1}^T p_\theta(x_t \mid x_{<t}), \quad \mathcal{L}_{\text{seq}} = - \sum_{t=1}^T \ln p_\theta(x_t \mid x_{<t}).$$

The first prediction needs an initial state or beginning-of-sequence convention. A recurrent state and a causally masked attention mechanism can both implement this factorization, but their code must expose only the allowed information.[6]

Input position j predicts the NEXT base, not the current base.



Allowed for output j : inputs $0 \dots j$. Forbidden: inputs $j+1 \dots T-1$.

A diagonal-inclusive mask is causal only with this shifted target convention.

Figure 2.2: The implementation convention used here: input position j predicts the next base. With zero-based arrays, inputs are the full sequence except its last token; targets are the full sequence except its first. Output j may use inputs 0 through j , but not input $j+1$, which contains its answer.

For a full array $z_0, \dots, z_T$, set $u_j = z_j$ and $y_j = z_{j+1}$, for $0 \leq j < T$. Then output j may attend to input j without leakage: that input is the previous symbol relative to its target. If targets were instead z_j at the same position, allowing access to u_j would reveal the answer. Thus a lower-triangular mask cannot be judged apart from target indexing.

Teacher forcing supplies true earlier symbols during training. It does not authorize true future symbols. Padding masks, packed-sequence boundaries, state reset, preprocessing, pooling and retrieval can all violate the information contract even when the principal attention mask looks correct. A causal architecture family is not a proof that its surrounding pipeline is causal.

2.4 Tokenization changes the prediction problem

For canonical DNA strings, $\Sigma = \{A, C, G, T\}$. Real records can contain ambiguous bases; an experiment must declare whether they are excluded, masked, or represented with additional symbols. This chapter's executable examples reject anything outside the four-symbol alphabet.

Encoding of A C G T A C	Tokens	What changes
single bases	A / C / G / T / A / C	6 tokens; 4-symbol alphabet
overlapping 3-mers	ACG / CGT / GTA / TAC	next token shares 2 bases
nonoverlapping 3-mers	ACG / TAC	2 tokens; up to 64 types
illustrative segmentation	AC / G / TAC	variable lengths; learned rules

Figure 2.3: Several tokenizations of the same six-base string. The variable-length segmentation is illustrative, not a fitted BPE vocabulary. Overlapping three-mers share two bases, so next-token prediction contains a built-in overlap advantage.

Single-base tokens keep the vocabulary small and retain one position per nucleotide; HyenaDNA is an example of work using single-nucleotide resolution.[15] Fixed k -mers can have up to 4^k types. Stride matters: nonoverlapping k -mers shorten the sequence, whereas overlapping windows retain many positions and repeat information. DNABERT-2 illustrates genomic BPE tokenization,[16] but its masked-language task must not be treated as interchangeable with our autoregressive objective.

Consider iid uniform DNA and overlapping three-mers. Given ACG, the next window must begin CG; only its last base is new. Its conditional entropy is $\ln 4$, not $\ln 64$. A predictor can look good on this task by exploiting overlap alone, with no biological structure learned. This is not necessarily implementation leakage: it is a property of the declared representation and task. It becomes misleading when interpreted as evidence of long-range genomic understanding.

For variable-length tokens, comparing perplexities across tokenizers is generally invalid. The same raw string produces different numbers of prediction events. Report the summed negative log probability and the number of raw bases represented under a specified reversible coding and boundary convention. Tokenization rules must be fitted using permitted training data, not inferred from the held-out corpus.

2.5 Data families answer different questions

Natural genomic sequences test statistical structure in biological material under a chosen sampling distribution. Synthetic genomic-like sequences test designed regularities whose resemblance to nature must be stated. Independent random sequences are a sanity control. Algorithmic sequences isolate exact computation. Natural language, code and agent trajectories provide different kinds of context, targets and feedback.

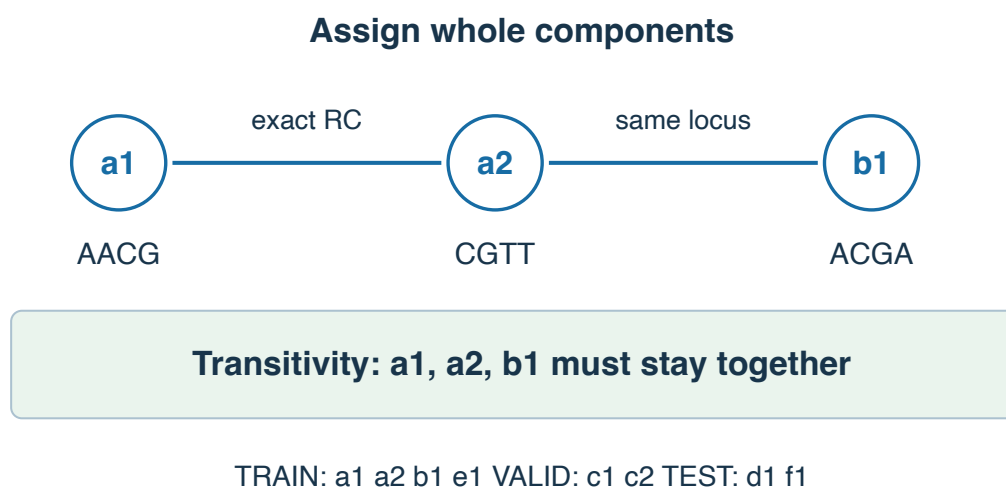
A motif task may measure local recognition while saying little about long-term memory. A delay task can stress exact retrieval without biological realism. Agent trajectories introduce action-dependent observations and may require policy evaluation rather than ordinary iid classification. Good evaluation does not force these data types into one score; it states which uncertainty each one reduces.

Tokenizing genomic text in a digital network is not itself molecular computation. The companion DNA Computing book studies physical molecular representations and operations. Here nucleotide strings are data processed by software. Genomic architecture terminology does not supply evidence of biological fidelity, cross-domain transfer or general intelligence.

2.6 Split relationships before splitting rows

Random row assignment is unsafe when rows are related. In genomics, possible relations include overlapping coordinates, exact duplicates, reverse complements, homologous sequences, shared loci, species and phylogenetic proximity. The appropriate exclusion relation depends on the claim: unseen-window interpolation within a known genome is not the same question as transfer to an unseen species.

Similarity-aware splitting methods such as SpanSeq make this issue explicit.[14] No universal identity threshold guarantees independence for every biological task. A chromosome-held-out split can prevent coordinate overlap yet still leave homologous sequences across folds. A species split can still include close evolutionary relatives.



Guarantee covers only detected/declared relations, not unknown homology.

Figure 2.4: Conflict relations must be closed transitively before assignment. Record a1 is the exact reverse complement of a2; a2 and b1 share declared locus metadata. All three must remain together even though a1 and b1 need not match exactly. The displayed fold assignment is actual output of the reference splitter.

Construct a conflict graph $H = (R, C)$ over records. An edge means the two records must not cross folds. If folds have no conflict edge between them, every connected component lies entirely within one fold: along a path, membership must remain equal at each edge. This is why independently grouping duplicates and loci after an initial split is insufficient.

Our splitter unions two relations: exact/reverse-complement sequence identity, using $\min(s, RC(s))$, and explicitly declared metadata groups. It assigns complete connected components with a deterministic seed-dependent order. Sorting stable IDs makes the result invariant to input row order. Large components can prevent exact target fold ratios; the code rejects fewer than three components rather than pretending to create three independent nonempty folds.

The guarantee is intentionally narrow. The code does not detect homology, coordinate overlap or phylogenetic proximity. Those relations must be computed or supplied before splitting. Sequence-specific preprocessing, augmentation and tokenizer training must also respect the final folds. A perfect conflict-graph algorithm cannot enforce an edge that was never recorded.

2.7 The random-data entropy control

Let the next target Y be independent of the permitted context C and uniform on V symbols. For any predictive distribution $q(\cdot | C)$ that depends only on permitted information,

$$\mathbb{E}[-\ln q(Y | C)] = \ln V + \mathbb{E}_C[D_{\text{KL}}(U_V \| q(\cdot | C))] \quad (2.2)$$

$$\geq \ln V. \quad (2.3)$$

This follows by adding and subtracting $\ln(1/V)$ in the expected log score. If q assigns zero probability to a possible target, its loss is infinite, which does not violate the bound. For iid uniform DNA, the optimum is $\ln 4 \approx 1.386294$ nats/base, or exactly two bits/base.

Independent uniform next base

Expected loss = $\ln 4 + \mathbb{E}[\text{KL}(\text{uniform} // \text{predicted distribution})]$

Valid reference

$\ln 4 = 1.386294$ nats/base
PPL = 4
BPB = 2

Deliberate future leak

0.001006 nats/base
accuracy = 1
invalid information access

Entropy constrains expected fresh-target loss, not every finite sample or training set.

Figure 2.5: The iid entropy control separates expected valid prediction from invalid information access. Uniform logits give exactly $\ln 4$ loss for every supplied canonical target. The deliberately future-reading predictor has much smaller measured loss because the answer is available to it.

The bound is about expectation on fresh independent targets. A finite test sample can favor a nonuniform predictor by chance. Repeated selection on that test sample can exploit its noise. A network can also memorize random training labels,[17] which is compatible with being unable to predict new independent labels. Training loss below $\ln 4$ is therefore not, alone, a causality diagnosis.

A sustained large advantage on genuinely fresh iid data should trigger an audit of target shift, future access, overlapping-token construction, reuse of random sequences, class frequencies, and scoring masks. The entropy control is a falsification instrument under explicit assumptions, not a theorem that natural genomes are incompressible.

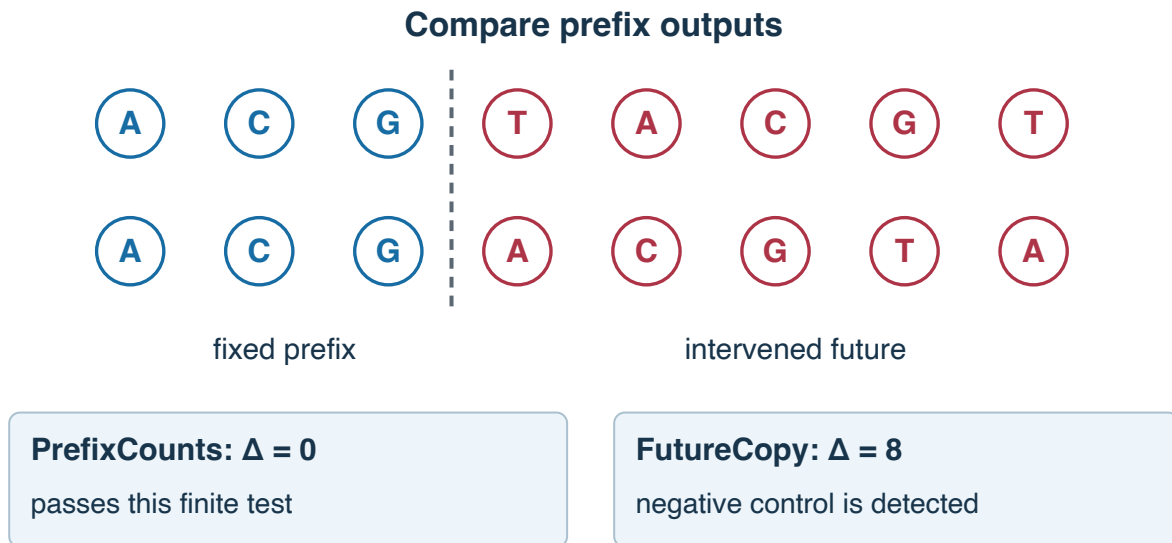
2.8 Intervene on future tokens

Suppose two inputs agree through position t . A causal output sequence should satisfy

$$f(u)_{\leq t} = f(u')_{\leq t} \quad \text{whenever} \quad u_{\leq t} = u'_{\leq t},$$

with suitable numerical tolerance in an implementation. Change only the suffix, recompute, and measure

$$\Delta_t = \max |f(u)_{\leq t} - f(u')_{\leq t}|.$$



Seven boundaries tested. Passing finite interventions is not a proof for all inputs.

Figure 2.6: A finite behavioral intervention. The prefix remains fixed, the future changes, and prefix logits are compared. The displayed toy results use deterministic float64 CPU references; every tested boundary gives zero for PrefixCounts and eight for FutureCopy.

The repository already provides `prefix_intervention_error`. It adds one modulo vocabulary size to every suffix token, runs the model in evaluation mode, and compares prefix logits. This chapter hardens that helper: nonfinite values cannot pass, malformed outputs are rejected, and original per-module train/eval modes are restored even if evaluation raises.

The helper is intended for deterministic, stateless forward calls of the supplied reference models. A stateful runtime must reset or clone its state for the two calls. Stochastic modules require controlled randomness or a distributional comparison; disabling dropout is not a cure for arbitrary hidden randomness. For large mixed-precision models, tolerance must be justified by the numeric path, scale and a no-intervention control, not chosen after seeing a violation.

The following wrapper tests every nontrivial boundary. It is extracted from `examples/deep/ch02.py`; the full source supplies its imports and the shared helper. The float64 reference tolerance is 10^{-12} , not a universal tolerance for other model implementations.

```

1 def intervention_suite(model, ids, atol=1e-12):
2     if ids.ndim != 2 or ids.shape[0] < 1 or ids.shape[1] < 2:
3         raise ValueError("nonempty batch and prefix/future required")
4     if ids.dtype != torch.long or ids.min() < 0 or ids.max() >= 4:
5         raise ValueError("DNA token indices required")
6     if not math.isfinite(atol) or atol < 0:
7         raise ValueError("finite nonnegative tolerance required")
8     # The existing helper performs +1-mod-V suffix interventions and restores mode.
9     errors = [prefix_intervention_error(model, ids, prefix_length=t, vocab_size=4)
10               for t in range(1, ids.shape[1])]
11     if not all(math.isfinite(e) for e in errors):
12         raise ValueError("non-finite intervention output cannot pass")
13     return {"errors": errors, "maximum": max(errors), "passes": max(errors) <= atol,
14            "atol": atol, "dtype": "float64 reference on CPU"}

```

Passing a finite set of interventions is necessary evidence, not a universal proof. A noncausal function can be insensitive to the particular suffix change; for example, it may depend on a suffix property preserved by that transformation. Test different suffixes, boundaries, sequence lengths, batches and masking modes, and inspect the computational graph. A deliberately leaky negative control establishes that the test can detect at least one relevant failure.

2.9 Why gradients alone are not enough

A derivative such as $\partial f_t / \partial u_{t+1}$ probes local sensitivity where that derivative exists. Tiny weights, saturation, quantization, discrete branching, detached tensors and poorly conditioned numeric paths can make a local gradient zero or uninformative while finite future changes alter behavior.

For a simple counterexample, a function can return one value when a future token equals A and another otherwise. The token ID is discrete; a derivative with respect to that ID is not the behavior we need to test. A detached future-dependent tensor provides another software counterexample: automatic differentiation sees no gradient path even though the forward result still depends on the future.

Finite intervention directly tests the relevant behavior, but it too samples only selected cases. The strongest causal argument combines a structural information-flow proof with executable interventions and negative controls. Neither a small gradient nor one zero difference should be promoted into an unconditional guarantee.

2.10 A CPU reference experiment with a known failure

The registered child record EV0-EXP01-CH02-SM0KE extends the existing EVO-EXP01 pipeline experiment. Its question, seeds and falsifiers were written before execution. No optimizer steps, trained DOGMA model, Hermon model, or GPU run are involved.

Run `python examples/deep/ch02.py` from the repository root with PyTorch installed. The recorded reference environment uses torch 2.10.0 on CPU. The JSON output includes individual seeds, split membership, metrics, intervention differences, an exact delay example and the specification hash.

For each seed 11, 29 and 47, generate eight independent sequences of 33 uniform DNA tokens. Inputs and next-base targets each have 32 positions. Uniform logits score all 256 targets at exactly $\ln 4$. PrefixCounts uses cumulative observed base counts plus one pseudocount per class and emits their logarithms. Softmax normalizes these into a predictive distribution; the reference has no fitted parameters.

The deliberately invalid FutureCopy reads input $j + 1$ to predict target j and places logit 8 on that symbol, zero elsewhere. Its probability on the target is

$$\frac{e^8}{e^8 + 3}, \quad \mathcal{L} = \ln(1 + 3e^{-8}) \approx 0.001005882.$$

The last output is excluded because no next input token exists there. Consequently this negative control scores 248 targets, while the ordinary baseline summaries score 256. These are diagnostic displays, not a fair model ranking; comparative averages must use the same mask and denominator.

Seed	Uniform nats/base	PrefixCounts nats/base
11	1.386294	1.462666
29	1.386294	1.419432
47	1.386294	1.454510

The finite-intervention suite tests seven boundaries on a fixed eight-token input. PrefixCounts gives $\Delta_t = 0$ at every boundary; FutureCopy gives $\Delta_t = 8$. Excellent leaked-target loss and failed causality are therefore visible in the same executable example.

The result artifact records the specification hash and torch version. Seeds here vary synthetic evaluation data, not training initialization. The parent EVO-EXP01 training study remains NOT RUN. Reproducing a CPU reference diagnostic is a smaller achievement than reproducing a trained-model benchmark, and it is useful precisely because its truth conditions are explicit.

2.11 Capacity is a vector of confounds

Equal parameter count does not imply equal representational width, memory access, train-time work or inference state. Report trainable parameters, embedding width, depth, state dimension, attention heads, KV heads, FFN width, context length and cache/state precision where applicable. Also report shared or tied parameters and any externally retrieved memory.

Matched quantity	Confound controlled	Still uncontrolled
parameters	stored trainable scalars	state, width, data, FLOPs
width	representation dimension	depth, parameters, history
state / cache bytes	runtime memory budget	precision, access semantics
training tokens	data exposure	compute per token
training FLOPs	arithmetic budget	hardware utilization
wall time	time on specified system	optimization / kernel maturity

Figure 2.7: Different matching strategies control different variables. Parameter matching, width matching, state/cache matching, token matching, compute matching and wall-time matching are separate experimental choices, not interchangeable definitions of fairness.

For an intentionally stripped-down example with vocabulary V , width d , untied input/output embeddings and no biases, a single recurrent matrix gives

$$P_{\text{rec}} = 2Vd + d^2.$$

A toy attention block with four d -by- d projections and a two-matrix FFN of width m gives

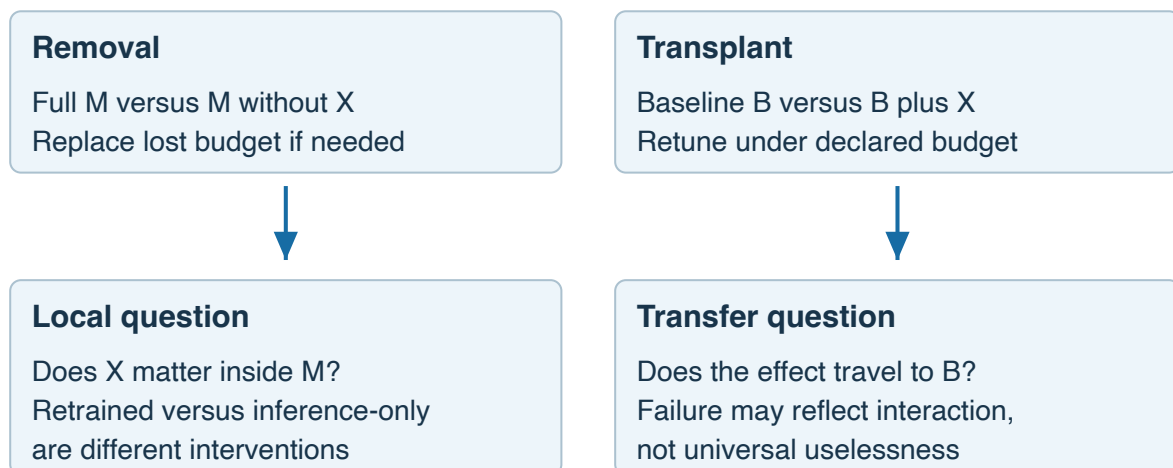
$$P_{\text{attn}} = 2Vd + 4d^2 + 2dm.$$

At $V = 4, d = 16, m = 32$, the counts are 384 and 2,176. These are declared toy formulas, not the parameter counts of a complete GRU, DOGMA model or Transformer. Their purpose is to show how the same width leaves different parameter budgets.

Conversely, solving for widths under one common parameter budget produces different representation dimensions. Fixed recurrent state and addressable KV history also provide different memory interfaces even when their byte budgets match. A useful study therefore uses more than one controlled comparison and reports the residual mismatches rather than claiming one scalar has removed every confound.

2.12 Baselines, removal tests and transplants

Begin with a trivial baseline: uniform class probabilities or the majority class under the correct split. Add a task-aware heuristic, such as smoothed prefix frequencies. Then include standard architecture baselines with competent tuning. Potential DOGMA comparisons include GRU/LSTM, state-space, Mamba-like and RWKV-like families; potential Hermon DNA comparisons include ordinary Transformers and relevant genomic models. A family's name does not establish that a particular implementation is strong or task-compatible.[10]



Use paired seeds and uncertainty for differences; neither contrast alone proves mechanism.

Figure 2.8: Removal and transplant experiments ask different questions. Removing a component tests its contribution inside a particular system. Adding it to another baseline tests whether an effect transfers. Budget replacement, retraining policy and paired uncertainty determine what either contrast can support.

Let Q be a higher-is-better metric and X a component. The difference $Q(M) - Q(M \setminus X)$ is interpretable only after defining the intervention. Removing X at inference from a trained system measures reliance under that disruption. Retraining without X measures performance of a different learning procedure. Replacing X with an equally sized generic block asks whether its structure matters beyond budget.

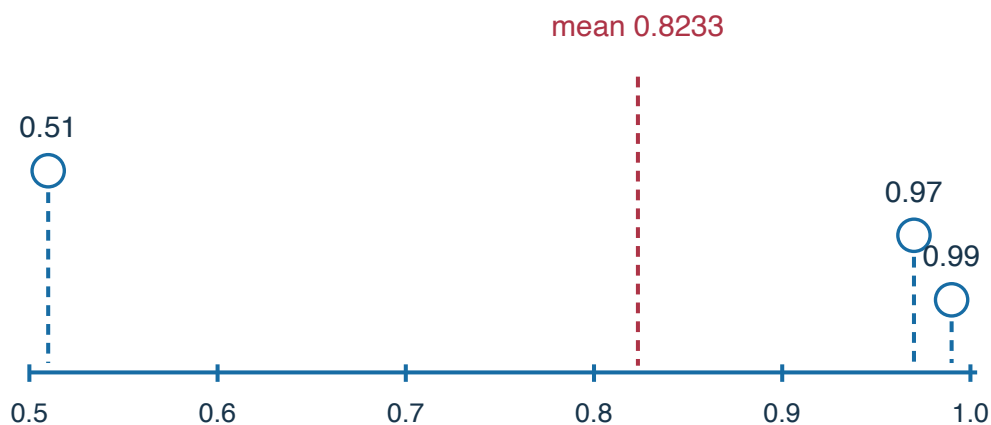
A transplant compares $B + X$ with B . A transferred improvement supports a portable contribution under those conditions. Failure to transfer can reflect an interaction with the original architecture, optimization or task; it does not prove universal uselessness. Use paired data splits and seeds when appropriate, and compute uncertainty on per-pair differences rather than comparing overlapping marginal error bars by eye.

2.13 Seeds reveal outcomes, not just a mean

For scores $m_1, \dots, m_k$, preserve individual values and report summaries suited to the question:

$$\bar{m} = \frac{1}{k} \sum_i m_i, \quad s = \sqrt{\frac{\sum_i (m_i - \bar{m})^2}{k - 1}}.$$

The sample standard deviation describes dispersion among these runs. It is not automatically a confidence interval for an architecture’s universal performance. Data sampling, initialization, optimizer stochasticity, tuning choices and hardware behavior can contribute different variance sources.[18]



Supplied illustrative values, not measured runs; three points do not establish bimodality.

Figure 2.9: The supplied illustrative values 0.51, 0.97 and 0.99 have mean 0.8233, median 0.97 and sample standard deviation about 0.2715. Their individual positions show what the mean hides. These are not measured training runs and are too few to establish bimodality.

One possible explanation is one failed optimization run and two successful runs, but the numbers alone do not prove that explanation. Inspect learning curves, failure traces and additional prespecified runs. If optimization failure is part of the method’s behavior, silently discarding that run changes the estimand. Distinguish infrastructure failures from valid but poor outcomes using rules established before results are seen.

Fixed seeds assist rerunning one environment; they do not guarantee identical outcomes across releases, devices or every nondeterministic operation.[21] Record the implementation, environment and input hashes as well as the seed. For genomic test uncertainty, resampling highly related windows independently can understate uncertainty; the resampling unit should reflect the data-generating relationships.

2.14 Loss, perplexity and bits: keep the denominator

For target distribution p and prediction q ,

$$H(p, q) = - \sum_x p(x) \ln q(x).$$

For observed target indices, empirical cross entropy is mean negative log probability. With N scored events, $\mathcal{L} = -N^{-1} \sum_i \ln q(y_i | c_i)$. Perplexity is $\exp(\mathcal{L})$: the exponential of the average surprise, not an average of per-batch perplexities.

If one event represents one base, bits per base are $\mathcal{L}/\ln 2$. More generally, if scored tokens represent B raw bases,

$$\text{BPB} = \frac{-\sum_i \ln q(y_i | c_i)}{B \ln 2}.$$

Boundary symbols, padding, ambiguous bases and overlapped encodings need a declared convention. An overlapping-window token score is not automatically a lossless coding rate per raw base.

Suppose one batch has ten scored positions with mean loss 1 and another has ninety with mean loss 2. The combined loss is $(10 + 180)/100 = 1.9$, not 1.5. Compute total negative log likelihood and total valid-target count, then divide and exponentiate. For distributed evaluation, aggregate both totals across workers before the final conversion.

The reference accepts finite logits of shape $[B, T, V]$ and integer targets of shape $[B, T]$. PyTorch cross entropy applies the log-softmax internally; passing normalized probabilities as though they were logits changes the computation.^[20]

```

1 def token_metrics(logits, targets):
2     """Unweighted next-base loss; all supplied positions are scored."""
3     if logits.ndim != 3 or targets.shape != logits.shape[:2] or targets.numel() == 0:
4         raise ValueError("expected nonempty logits [B,T,V] and targets [B,T]")
5     if not torch.isfinite(logits).all():
6         raise ValueError("non-finite logits")
7     if targets.dtype != torch.long or targets.min() < 0 or targets.max() >= logits.shape[-1]:
8         raise ValueError("target indices outside vocabulary")
9     loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]), targets.reshape(-1))
10    nats = float(loss)
11    return {"nats_per_base": nats, "perplexity": math.exp(nats),
12            "bits_per_base": nats / math.log(2),
13            "accuracy": float((logits.argmax(-1) == targets).double().mean())}

```

This small function scores every supplied position and has no padding argument. A production evaluator needs an explicit valid-target mask and a declared empty-mask policy. Returning a convenient zero for no scored targets would make a missing evaluation look perfect.

2.15 Accuracy, calibration and systems measurements

Accuracy counts correct argmax decisions; it does not distinguish confidence 0.51 from 0.99 on a correct binary decision. For imbalanced classification, precision and recall separate false-positive and false-negative behavior, and

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

requires a declared policy when the denominator is zero. Macro, micro and per-class reporting answer different aggregation questions. Thresholds and any calibration transform must be chosen using permitted development data, not optimized on the final test labels.

Calibration asks whether stated probabilities correspond to observed frequencies under the evaluated population. It is distinct from accuracy;^[19] a high-accuracy classifier can be overconfident. Calibration summaries also depend on binning, sample size and distribution shift, so report their construction rather than treating one number as an invariant property.

Evidence axis	Measure	Required context
predictive quality	NLL, BPB, F1, retrieval accuracy	same task / targets / split
confidence quality	calibration, proper log score	population + fitted calibration
systems performance	latency, throughput, memory	hardware / batch / precision
engine fidelity	reference-output difference	same weights / inputs / dtype

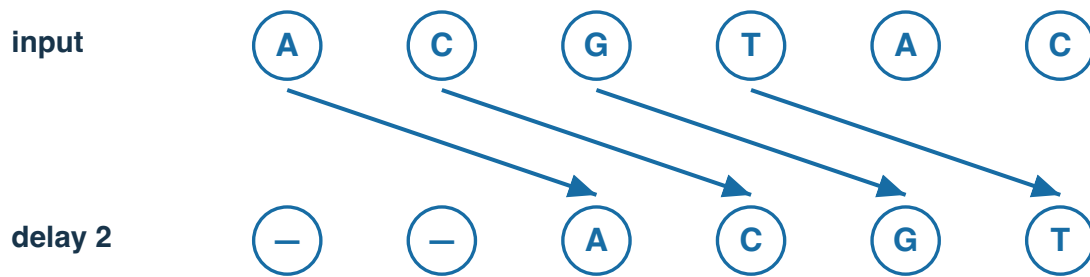
Figure 2.10: Predictive quality, confidence quality, systems performance and engine fidelity are distinct axes. A faithful optimized engine can execute a weak model correctly; a strong model can be executed incorrectly. Neither capability nor parity can be inferred from speed alone.

Retrieval accuracy should specify what is retrieved, top- k versus exact match, and the candidate pool. Latency needs batch size, sequence lengths, prefill versus decode, synchronization and warm-up policy. Throughput needs a unit such as scored bases or generated tokens per second. Runtime memory needs peak versus steady-state and inclusion of weights, cache, workspace and allocator overhead.

Report a vector such as (Q, L, U, R) for quality, latency, throughput and runtime memory. Engine parity is an additional correctness axis: compare the same weights and inputs against a trusted implementation under justified numeric tolerances. Do not infer latency from a parameter formula or an architecture label; this chapter runs no performance benchmark.

2.16 Exact synthetic tasks before learned solutions

An algorithmic task should have an exact oracle before training. For delay ℓ , define $y_t = x_{t-\ell}$ only at positions $t \geq \ell$. The first ℓ positions are unscored, not assigned a convenient default label.



Zero-based scored positions 2...5. First two positions are unscored, not default-A targets.

Figure 2.11: Delay-two oracle on six inputs. Each scored target points to an exact earlier symbol. The first two outputs are excluded because their source positions do not exist. This is a past-retrieval task, not next-base language modeling.

For ACGTAC and lag two, scored positions 2,3,4,5 have targets ACGT. On iid four-symbol inputs, a context-free guess has expected accuracy $1/4$ on these targets, while the exact oracle reaches one. The full-history Bayes predictor can be perfect, so the random next-base entropy lower bound does not apply to this different task.

Test lag one, near-maximum lag, repeated symbols, invalid lag, different lengths and target alignment. An oracle is necessary but not sufficient: its generator can still leak labels through length, frequency, position or output history. A balanced binary parity task, for example, should not accidentally make one class dominate through its sampling scheme.

2.17 Balanced memory tasks and shortcut audits

Task	Required memory	Shortcut to exclude
exact delay / copy	recover specified past symbol	fixed position / output leak
parity / running sum	compressed statistic	class imbalance / length
associative retrieval	key-conditioned lookup	key frequency / duplicates
pointer chasing	sequential dependent lookup	one-hop proxy
motif / conditional routing	pattern and state	future context / motif count

Figure 2.12: A memory suite separates exact retrieval, compressed statistics, associative lookup, dependent lookup and local/state-conditioned recognition. Different tasks expose different memory interfaces. No result for one row establishes superiority on the whole matrix.

Copy and delay test recovery of specified past symbols. Parity and running sum test compact sufficient statistics. A finite-state-machine task tests state transitions. Associative retrieval tests key-conditioned recall and must define duplicate-key semantics, such as first-write, last-write or unique-key-only. Pointer lookup is one addressed retrieval; pointer chasing requires repeated dependent retrievals and should vary chain depth. Motif recognition and conditional routing test pattern-dependent behavior but can admit count or position shortcuts.

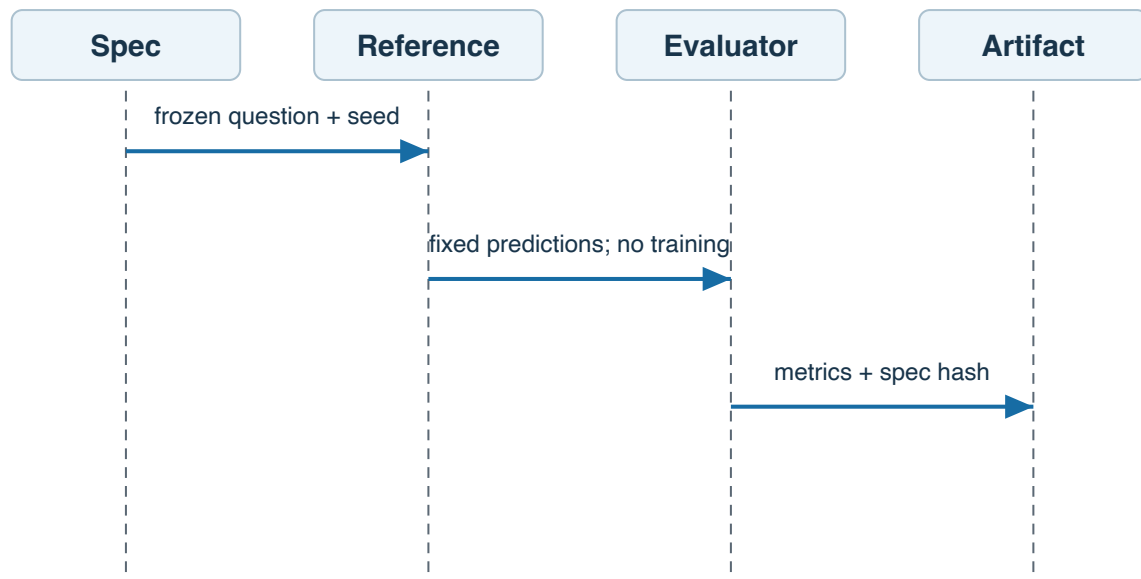
For each generator ask whether position, sequence length, token frequency, reused examples, output history or future context reveals the answer. Permuting nuisance positions while preserving the correct answer can expose position shortcuts. Balancing labels conditional on length can remove a length proxy. Holding out key identities or relationship components tests a different claim from holding out randomly generated rows.

DOGMA-style carried state and Hermon-style addressable history should face both compressed-statistic and exact-retrieval tasks. A fixed-state model may represent a running statistic compactly without retaining every token. A history-addressable model may retrieve an old symbol conveniently while paying cache costs. Testing only the task favored by one interface would answer a narrower question than a general memory comparison.

For length extrapolation, train at lengths up to T_{train} and evaluate beyond it, but specify what changes: absolute delay, relative delay, number of keys, pointer depth, distractors or output length. Keeping delay fixed while increasing irrelevant suffix length is not the same stress as increasing retrieval distance. One successful synthetic extrapolation does not establish general intelligence.

2.18 Budgets and the experiment lifecycle

Before a model comparison, fix or report tokens seen, optimizer steps, batch size, optimizer and learning-rate schedule, tuning budget, hardware, precision and stopping rule. Equal steps with different batch sizes are not equal data exposure. Equal tokens are not equal FLOPs. Equal wall time can confound architecture with implementation maturity.



UML-style sequence for EVO-EXP01-CH02-SMOKE. Parent training remains NOT RUN.

Figure 2.13: UML-style sequence for the actual Chapter 2 child record: a frozen specification drives fixed reference predictions, evaluation and a hashed result artifact. There is no trainer message because this diagnostic performs no training. The parent training study remains unrun.

The existing research program defines EVO-EXP01 through EVO-EXP05. This chapter adds a scoped child to EVO-EXP01, not a competing experiment registry. Its question is whether the reference pipeline distinguishes causal prediction from deliberate future access while keeping declared record conflicts inside folds. Falsifiers include a nonzero causal difference above the declared reference tolerance, an undetected negative control, cross-fold declared conflicts, and accepting nonfinite outputs.

Freeze the specification before running, and record amendments as new versions rather than rewriting the initial hypothesis to fit the observed result. A hash binds a result to exact specification bytes; it does not prove the specification was scientifically wise. Mechanism claims still need valid tasks, controls, measurements and skeptical alternatives.

The next chapter is *Differentiation, optimization and tensor programs*. It derives how parameters are updated and how tensor computations implement those updates. Later chapters can then introduce sequence models without losing the distinction between a new mechanism, a correctly executed program and convincing evidence of improvement.

2.19 Exercises

1. Derive Equation 2.3. Explain why it does not prohibit fitting a finite random training set.
2. Write inputs and targets for ACGTA. Which input positions may output 2 use under this chapter's zero-based convention?
3. Derive the next-token entropy of overlapping three-mers on iid uniform DNA. Compare with nonoverlapping three-mers.
4. Build a three-record transitive conflict example. Explain why splitting exact duplicates and metadata groups independently is insufficient.

5. Implement a future-dependent function with zero automatic gradient to its future input. Which finite intervention would detect it?
6. Why might adding one modulo four to every suffix token miss a future-dependent function? Propose an additional intervention.
7. At $V = 4, d = 16, m = 32$, derive the two toy parameter counts. List three remaining confounds after matching either count.
8. Interpret the supplied values 0.51, 0.97 and 0.99. What additional evidence is needed before calling the distribution bimodal?
9. Combine ten scored targets with mean loss 1 and ninety with mean loss 2. Convert the aggregate to perplexity and BPB for base tokens.
10. Define an associative-retrieval oracle with duplicate keys and audit two shortcuts. Specify an extrapolation test.
11. Design a removal and a transplant experiment for a proposed routing component, including budgets and a falsifier.
12. A fast engine differs from reference logits while the model's task accuracy is high. Which claims remain supported, and which fail?

2.20 Worked solutions

1. Conditional on each permitted context, add and subtract $-\ln(1/V)$ in the uniform target expectation. The residual is $D_{\text{KL}}(U_V \| q)$, which is nonnegative. Memorization uses information about already observed training targets; the bound concerns independent fresh targets under the allowed context.
2. Inputs are ACGT, targets CGTA. Output 2 predicts T and may use input positions 0,1,2, namely A,C,G. Input position 3 is T and would reveal that target. A diagonal-inclusive causal mask is consistent with this shift.
3. Given the current overlapping three-mer, two bases of the next token are known and one is uniform, so conditional entropy is $\ln 4$. A fresh nonoverlapping three-mer contains three independent bases and has entropy $3 \ln 4 = \ln 64$. The token-level numbers concern different amounts of new information.
4. Let a_1 and a_2 be reverse complements and a_2 and b_1 share a locus. The first relation requires a_1 and a_2 together; the second requires a_2 and b_1 together; hence all three belong together. Assigning the two group systems independently can satisfy one relation while violating the other.
5. A forward computation using a detached future embedding can affect the output while cutting the automatic-gradient path. Change the actual future token and recompute the prefix output from a reset state. The finite output difference, not the detached gradient, exposes the dependence.
6. A function depending on equality of two future tokens is unchanged when both are incremented modulo four. Change only one suffix position, use independently resampled suffixes, and vary intervention lengths. Even these are finite checks; structural analysis is still needed.

7. Embeddings contribute $2Vd = 128$. The recurrent matrix adds 256, totaling 384. Four attention projections add 1,024 and the FFN adds another 1,024, totaling 2,176. Matching parameters leaves memory access, state/cache bytes, depth, optimization and compute per token uncontrolled.
8. The mean is 0.8233, median 0.97 and sample standard deviation about 0.2715. One poor outcome is visible, but a three-point sample cannot establish the generating distribution's modes or cause. More prespecified runs, learning curves and failure diagnostics are needed.
9. Aggregate loss is 1.9 nats/base. Perplexity is $e^{1.9} \approx 6.686$ and BPB is $1.9 / \ln 2 \approx 2.741$. The loss exceeds the uniform-DNA reference because these hypothetical predictions are worse calibrated to their targets. Exponentiating each batch first and averaging gives a different, inappropriate aggregate.
10. One valid rule is last-write-wins: scan key–value writes in order, overwrite that key's stored value, then return the final value for the query key. Generate duplicate keys deliberately and define missing-query behavior. Balance query identity and position; prevent value frequency from predicting the answer. Extrapolate number of stored keys and retrieval distance separately.
11. Compare a full system with a retrained budget-matched generic routing replacement, using paired splits/seeds and equal declared tuning resources. Separately add the proposed component to a standard baseline with a matched added budget. A prespecified lack of improvement on the mechanism-targeted task, or gains explained by a generic replacement, weakens the proposed mechanism claim.
12. High task accuracy supports only the evaluated model/protocol result, subject to its own controls. Engine parity fails if differences exceed justified tolerance on the stated contract. Speed can still be measured, but it does not establish a correct accelerated implementation. Test the engine on identical weights and inputs before attributing model behavior to faithful execution.

Chapter 1 glossary

Stored structure The persistent description available for future activity. See Section 1.2.

Expressed computation The ordered activity selected or constructed for a context. See Section 1.2.

Adapted structure A persistent description after a specified update process. See Section 1.2.

Gene expression Production of functional RNA or protein from genomic information. See Section 1.3.

Regulation Biological control of activity; in the proposed software interface, a context-dependent control decision. See Section 1.3.

Plan An ordered list of operations with defined composition semantics. See Section 1.4.

Development Biological construction processes; by explicit analogy, a software description-to-program map. See Section 1.5.

State migration A declared way to move or reset state when structure changes. See Section 1.6.

Novelty burden The need to identify a contribution beyond established alternatives. See Section 1.7.

DOGMA The non-Transformer DNA-native model and corresponding engine research line. See Section 1.8.

Hermon DNA The Transformer-based DNA model and corresponding engine research line. See Section 1.8.

Evolutor The broader research, compiler/runtime and orchestration framework above both families. See Section 1.8.

KV cache Stored key/value representations for retained context positions. See Section 1.9.

Parity Agreement with a specified reference behavior under declared tolerances. See Section 1.10.

Chapter 2 glossary

Information contract The inputs and state that a prediction is permitted to depend on, including preprocessing, retrieval and target alignment.

Conflict component A connected component of record relationships that prohibit cross-fold separation. Guarantees cover only the supplied or detected relations.

Finite intervention A behavior test that changes a selected input region and compares outputs required to remain invariant. Passing finitely many cases is not a universal proof.

Entropy control An analytical reference under a specified data distribution and information boundary; iid uniform DNA has next-base entropy $\ln 4$ nats.

Scoring denominator The number of valid events or represented raw bases used to normalize total negative log likelihood. Mask and tokenizer conventions matter.

Removal and transplant Interventions that remove a component from one system or add it to another. They test different causal explanations under declared budgets.

Engine fidelity Agreement of an implementation with a trusted numerical reference for the same model, weights and inputs; distinct from task capability.

Bibliography

- [1] National Human Genome Research Institute. Gene Expression. [Source link](#).
- [2] Bruce Alberts et al. An Overview of Gene Control. Molecular Biology of the Cell, 4th edition (2002). [Source link](#).
- [3] Scott F. Gilbert. Principles of Development: Genes and Development. Developmental Biology, 6th edition (2000). [Source link](#).
- [4] Scott F. Gilbert. Mechanisms for the environmental regulation of gene expression. Birth Defects Research C 72 (2004), 291–299. [Source link](#).
- [5] Noam Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (2017). [Source link](#).
- [6] Ashish Vaswani et al. Attention Is All You Need (2017). [Source link](#).
- [7] Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning (2017). [Source link](#).
- [8] Sumit Gulwani, Alex Polozov, and Rishabh Singh. Program Synthesis (2017). [Source link](#).
- [9] John R. Koza. Research overview: genetic programming. [Source link](#).
- [10] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces (2023; revised 2024). [Source link](#).
- [11] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models (2023). [Source link](#).
- [12] Ian Walsh et al. DOME: recommendations for supervised machine learning validation in biology. Nature Methods 18 (2021), 1122–1127. [Primary article](#).
- [13] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. JMLR 11 (2010), 2079–2107. [Primary article](#).
- [14] Alfred Ferrer Florensa et al. SpanSeq: similarity-based sequence data splitting method for improved development and assessment of deep learning projects. NAR Genomics and Bioinformatics 6 (2024), lqae106. [Primary preprint](#).
- [15] Eric Nguyen et al. HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution (2023). [Primary preprint](#).
- [16] Zhihan Zhou et al. DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome. ICLR (2024). [Primary preprint](#).
- [17] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht and Oriol Vinyals. Understanding deep learning requires rethinking generalization. ICLR (2017). [Primary preprint](#).
- [18] Xavier Bouthillier et al. Accounting for Variance in Machine Learning Benchmarks. MLSys (2021). [Primary article](#).

- [19] Chuan Guo, Geoff Pleiss, Yu Sun and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. PMLR 70 (2017), 1321–1330. [Primary article](#).
- [20] PyTorch contributors. CrossEntropyLoss, version 2.10 documentation. [Official documentation](#). Consulted 8 September 2026.
- [21] PyTorch contributors. Reproducibility, version 2.10 documentation. [Official documentation](#). Consulted 8 September 2026.

Index

ablation, 24
Adapted structure, 2
autoregression, 16

bits per base, 25

causality, 20
conflict component, 19
cross entropy, 25

data leakage, 19
Development, 6
DOGMA, 10

entropy control, 20
Evolutor, 10
Expressed computation, 2

finite intervention, 20

Gene expression, 3

Hermon DNA, 10

perplexity, 25

reverse complement, 19

Stored structure, 2

target alignment, 16
transplant test, 24